Vector<E>
----
public class Vector<E> extends AbstractList<E> implements List<E>, Cloneable, Serializable, RandomAccess

* It is a predefined **legacy class** of List<E> interface available from JDK 1.0V
* It is an implemented class of List<E>interface.
* It can accept duplicate, null, homeogenous and hetreogeneous elements.
* It stores the element based on the index position.
* It uses **Dynamic array data structure. [Object []]**
* **It is dynamically Growable in nature.**
* The initial default capacity is 10, next capacity would be 20 and so on.
* Methods are **synchronized** so performance wise It is slow than ArrayList.
* It is mainly used to perform **read operation (Retrieve Operation) where thread safety is required.**
* Here Iterators are **Fail-Fast Iterator so It will throw java.util.ConcurrentModificationException.**
* hashCode() & equals() methods are overridden because It follows the contract.
* It implements from RandomAccess marker interface (JDK 1.4V) so can select the element randomly based on the index position

Constructors :

We have 4 types of Constructor in Vector

1) Vector v1 = new Vector();
   It will create the vector object with default capacity is 10.

2) Vector v2 = new Vector(int initialCapacity);
   Will create the vector object with user specified capacity.

3) Vector v3 = new Vector(int initialCapacity, int capacityIncrement);
   Eg :- Vector v = new Vector(100,5);

   Initially It will create the Vector Object with initial capacity 100 and then when the capacity will be full then increment by 5 so the next capacity would be 105, 110 and so on.

4) Vector v4 = new Vector(Collection c);
   We can achieve loose coupling

Vector Programs :

```java
package com.ravi.vector_demo;

import java.util.Vector;

public class VectorDemo1
{
    public static void main(String[] args)
    {
        Vector<Integer> listOfNumbers = new Vector<>();
        listOfNumbers.add(100);
        listOfNumbers.add(200);
        listOfNumbers.add(300);
        listOfNumbers.add(400);
        listOfNumbers.add(500);
        listOfNumbers.add(600);


        //Finding the sum of all the elements
        int sum = 0;
        for(Integer num : listOfNumbers)
        {
            sum += num;
        }

        IO.println("Sum of all the elements :"+sum);

    }

}
```

Collection with Custom Object :

```java
package com.ravi.vector_demo;

import java.util.Collections;
import java.util.Vector;

record Product(Integer id, String name, Double price) implements Comparable<Product>
{
    @Override
    public int compareTo(Product p2)
    {
        return Integer.compare(this.id(), p2.id());
    }

}

public class VectorDemo2
{
    public static void main(String[] args)
    {
        Vector<Product> listOfProduct = new Vector<>();
        listOfProduct.add(new Product(222,"Laptop",98000D));
        listOfProduct.add(new Product(111,"Mobile",54000D));
        listOfProduct.add(new Product(333,"Camera",72000D));

        IO.println("Original Object :");
        for(Product product : listOfProduct)
        {
            IO.println(product);
        }

        Collections.sort(listOfProduct);

        IO.println("Sorting Object based on the Id (Ascending) :");
        for(Product product : listOfProduct)
        {
            IO.println(product);
        }

        IO.println("Sorting Object based on the Id (Descending) :");
        Collections.sort(listOfProduct, (p1,p2)-> p2.id().compareTo(p1.id()));
        for(Product product : listOfProduct)
        {
            IO.println(product);
        }
    }
}
```

Collections.sort(List<E> list)
Collections.sort(List<E> list, Comparator<E> cmp)

```java
package com.ravi.vector_demo;

import java.util.Vector;

public class VectorDemo3 {

    public static void main(String[] args)
    {
        Vector<Integer> listOfNumber = new Vector<>(100, 10);
        IO.println("Initial Capacity is :"+listOfNumber.capacity());

        for(int i=0; i<100; i++)
        {
            listOfNumber.add(i);
        }

        IO.println("After adding 100 elements :");
        IO.println("Size is  :"+listOfNumber.size());
        IO.println("Capacity is  :"+listOfNumber.capacity());

        listOfNumber.add(999);
        IO.println("After adding 999 element :");
        IO.println("Size is  :"+listOfNumber.size());
        IO.println("Capacity is  :"+listOfNumber.capacity());

    }

}
```

Note : If we know the size of array (Rough Idea) then better to use initiallCapacity parameter to prevent copy-paste from one object to another object as shown in the above program

---

Collections class method :

All the methods are static method only.

1) public static Comparator<E> reverseOrder() : Will reverse the collection.

2) public static void max(Collection<E> coll) : Will return max element

3) public static void min(Collection<E> coll) : Will return min element

```java
package com.ravi.vector_demo;

import java.util.Collections;
import java.util.Vector;

public class VectorDemo4 {

    public static void main(String[] args)
    {
        Vector<Integer> listOfNumber = new Vector<>();
        listOfNumber.add(67);
        listOfNumber.add(34);
        listOfNumber.add(99);
        listOfNumber.add(12);
        listOfNumber.add(29);

        IO.println("Original Data :"+listOfNumber);

        Collections.sort(listOfNumber, Integer::compareTo);
        IO.println("Ascending Order :"+listOfNumber);

        //Collections.sort(listOfNumber, (i1,i2)-> i2.compareTo(i1));
        Collections.sort(listOfNumber, Collections.reverseOrder());
        IO.println("Descending Order :"+listOfNumber);

        IO.println("Max Element :"+Collections.max(listOfNumber));
        IO.println("Min Element :"+Collections.min(listOfNumber));

    }

}
```

***What is Fail Fast Iteartor ?

[Modification is only allowed through Iterators own methods]

While iterating (retrieving) the object from the collection by using Itearor interface or for each loop OR ListIterator interface, if at any point of time the original structure is going to modify after the creation of Itearator by using any way except Iterators's own method [remove(), add(), set()] then It will throw an exception i.e. java.util.ConcurrentModificationExacption.

```java
package com.ravi.vector_demo;

import java.util.Vector;

record Employee(Integer id, String name, Double salary)
{

}

public class VectorDemo5
{
    public static void main(String[] args)
    {
        Vector<Employee> listOfEmployees = new Vector<>();
        listOfEmployees.add(new Employee(101, "Scott", 78890.90));
        listOfEmployees.add(new Employee(102, "Smith", 98890.90));
        listOfEmployees.add(new Employee(103, "John", 65890.90));
        listOfEmployees.add(new Employee(104, "Alen", 91890.90));
        listOfEmployees.add(new Employee(105, "Sanju", 99890.90));

        IO.println("Original Object");
        for(Employee employee : listOfEmployees)
        {
            IO.println(employee);
        }

        IO.println("Delete the employee object on the id :");

        Integer id = Integer.valueOf(IO.readln("Enter Employee Id :"));

        for(Employee employee : listOfEmployees)
        {
            if(employee.id().equals(id))
            {
                listOfEmployees.remove(employee);
                break;
            }
        }

        IO.println("After Deleting :");

        for(Employee employee : listOfEmployees)
        {
            IO.println(employee);
        }

    }

}
```

```java
package com.ravi.vector_demo;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Vector;

record Player(Integer id, String name, Double basePrice)
{

}

class IPLTeam
{
    private String name;
    private List<Player> listOfPlayers;

    public IPLTeam(String name)
    {
        this.name = name;
        listOfPlayers = new Vector<>();
    }

    public void addPlayer(Player player)
    {
        listOfPlayers.add(player);
    }

    public void displayAllPlayers()
    {
        for(Player player : listOfPlayers)
        {
            IO.println(player);
        }
    }

    public void retainOrRemove()
    {
         Iterator<Player> iterator = listOfPlayers.iterator();
         while(iterator.hasNext())
         {
             Player player = iterator.next();

             if(player.basePrice() >=5000)
             {
                 iterator.remove();
             }

         }
    }
}

public class VectorDemo6
{
    public static void main(String[] args)
    {
        IPLTeam ipl = new IPLTeam("SRH");
        ipl.addPlayer(new Player(101, "Abhishek", 5500D));
        ipl.addPlayer(new Player(102, "Ishan", 6500D));
        ipl.addPlayer(new Player(103, "Head", 4500D));
        ipl.addPlayer(new Player(104, "Klassen", 4800D));
        ipl.addPlayer(new Player(105, "Cummins", 4900D));
        ipl.addPlayer(new Player(106, "Nitish", 5000D));

        ipl.displayAllPlayers();

        ipl.retainOrRemove();

        IO.println("After retain OR remove, Final List :");

        ipl.displayAllPlayers();

    }

}
```